DESIGN AND IMPLEMENTATION OF AN ESP32-BASED WIRELESS INTRUSION DETECTION SYSTEM WITH TAMPER AND LINK SUPERVISION

A PROJECT REPORT

submitted by

EBS26016 LINGANIVETH S G

in partial fulfillment for the award of the degree of

MASTER OF TECHNOLOGY

IN

EMBEDDED SYSTEMS ENGINEERING

Course: 25ES604 Embedded System Design – Term Project



AMRITA SCHOOL OF ENGINEERING, COIMBATORE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE – 641112

NOVEMBER 2026

AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF ENGINEERING, COIMBATORE, 641112



BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"Design and Implementation of an ESP32-Based Wireless Intrusion Detection System with Tamper and Link Supervision"** submitted by **LINGANIVETH S G, EBS26016** in partial fulfillment of the requirements for the award of the degree of **Master of Technology in "EMBEDDED SYSTEMS"** is a bonafide record of the work carried out under my guidance and supervision at Amrita School of Engineering, Coimbatore.

<< Signature of the Supervisor >>
<< Name >>
SUPERVISOR
<< Designation >>
<< Department >>

<< Signature of the Chairperson >>
<< Name of Chairperson >>
CHAIRPERSON
<< Department >>

This project report was evaluated by us on …… (date)

<< Signature of the Internal Examiner with date >>
<< Name >>
INTERNAL EXAMINER
<< Designation >>
<< Organisation >>

<< Signature of the External Examiner with date >>
<< Name >>
EXTERNAL EXAMINER
<< Designation >>
<< Organisation >>

ACKNOWLEDGEMENT

I express my sincere gratitude to my project supervisor, [Name of Supervisor], [Designation], Department of [Department], for the guidance, encouragement and critical feedback given throughout this work, and for the emphasis on following the embedded system design flow, which shaped the way this project was planned and documented.

I thank [Name of Chairperson], Chairperson, Department of [Department], Amrita School of Engineering, Coimbatore, for providing the facilities and the opportunity to carry out this project as part of the course 25ES604 Embedded System Design.

I am grateful to the faculty members and laboratory staff of the department for their support with components and equipment, and to my classmates for their suggestions during the reviews.

Finally, I thank my family for their constant support and encouragement.

LINGANIVETH S G

ABSTRACT

Intrusion alarms are among the most widely deployed embedded systems, yet low-cost designs often fail silently: a cut sensor wire, a dead or out-of-range wireless detector, or an opened sensor enclosure goes unnoticed while the panel keeps reporting that everything is normal. This project presents HomeGuard, a two-node wireless intrusion detection system built on two ESP32 microcontrollers, developed by following the complete embedded system design flow: requirements, specification, system architecture, parallel hardware and software design with unit testing, system integration, validation, and operation and maintenance.

A battery-friendly detector node senses motion with an HC-SR501 PIR sensor, door opening with a magnetic reed switch and enclosure tampering with a lever microswitch. Switch inputs use fail-safe wiring with pull-up resistors and a two-layer debounce (a 1 ms RC filter and a 50 ms software window). Events are sent to the hub over ESP-NOW using a 12-byte packet with a sequence number and CRC-16; the hub acknowledges each event only after acting on it, the detector retries up to three times, and duplicates are filtered. A heartbeat every 5 s lets the hub declare the detector offline after 15 s of silence.

The hub implements a five-state alarm machine (disarmed, exit delay, armed, entry delay, alarm) with 30 s exit and 15 s entry delays, tamper detection in every state, a 4-digit PIN with lockout after three wrong attempts, a transistor-driven siren, status LEDs, an I2C OLED display and a web page served from its own Wi-Fi access point, so no router or internet is needed. Bench tests showed no packet loss in 452 consecutive exchanges with a typical round-trip time below 3 ms, and every requirement is traced to a hardware or software test. The complete design, firmware and documentation are maintained in a public Git repository so that the system can be reproduced.

Keywords: *ESP32, ESP-NOW, intrusion detection, PIR sensor, finite state machine, supervision, embedded system design flow*

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE No.
	ABSTRACT	ii
	LIST OF FIGURES	v
	LIST OF TABLES	vi
	LIST OF SYMBOLS	vii
	LIST OF ABBREVIATIONS	viii
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	1
	1.3 OBJECTIVES	1
	1.4 SCOPE AND CONSTRAINTS	2
	1.5 METHODOLOGY	2
	1.6 ORGANISATION OF THE REPORT	3
2	**BACKGROUND AND TECHNOLOGY SURVEY**	**4**
	2.1 INTRUSION ALARM SYSTEMS	4
	2.2 SENSING TECHNOLOGIES	4
	2.3 WIRELESS COMMUNICATION OPTIONS	5
	2.4 EMBEDDED DESIGN CONCEPTS APPLIED	5
3	**REQUIREMENTS ANALYSIS**	**6**
	3.1 STAKEHOLDERS AND USE CASES	6
	3.2 FUNCTIONAL REQUIREMENTS	6
	3.3 NON-FUNCTIONAL REQUIREMENTS	7
	3.4 DESIGN CONSTRAINTS	7
4	**SPECIFICATION**	**8**
	4.1 SYSTEM SPECIFICATION	8
	4.2 TIMING CONSTANTS	8
	4.3 INTERFACE SPECIFICATION	9
	4.3.1 Pin assignment	9
	4.3.2 Board identities	10
	4.4 MESSAGE SPECIFICATION	10
5	**SYSTEM ARCHITECTURE**	**12**
	5.1 HARDWARE/SOFTWARE PARTITIONING	12
	5.2 WHY TWO MICROCONTROLLERS	13
	5.3 WHY NO RTOS	13
	5.4 COMPONENT SELECTION	13
6	**HARDWARE DESIGN, IMPLEMENTATION AND TESTING**	**15**
	6.1 DETECTOR NODE DESIGN	15
	6.1.1 Fail-safe switch inputs	15

CHAPTER	TITLE	PAGE No.
	6.1.2 Debouncing	15
	6.1.3 PIR sensor interface	16
	6.2 HUB DESIGN	16
	6.2.1 Siren driver	17
	6.2.2 Status LEDs	17
	6.2.3 Keypad and display	17
	6.3 POWER BUDGET	17
	6.4 HARDWARE IMPLEMENTATION	18
	6.5 HARDWARE TESTING	19
7	**SOFTWARE DESIGN, IMPLEMENTATION AND TESTING**	**21**
	7.1 FIRMWARE STRUCTURE	21
	7.1.1 Interrupt and callback design	22
	7.2 COMMUNICATION PROTOCOL	23
	7.3 ALARM STATE MACHINE	24
	7.4 USER INTERFACE SOFTWARE	26
	7.4.1 Keypad and PIN handling	26
	7.4.2 OLED display	26
	7.4.3 Outputs and indications	26
	7.4.4 Phone web page	26
	7.5 IMPLEMENTATION	27
	7.6 SOFTWARE TESTING	28
8	**SYSTEM INTEGRATION AND VALIDATION**	**29**
	8.1 INTEGRATION PROCEDURE	29
	8.2 RESPONSE TIME MEASUREMENT	29
	8.3 RANGE TEST	30
	8.4 SYSTEM VALIDATION	30
	8.5 DEMONSTRATION SCENARIOS	31
	8.6 DISCUSSION	32
9	**OPERATION AND MAINTENANCE**	**33**
	9.1 USER GUIDE	33
	9.2 MAINTENANCE	33
	9.3 TROUBLESHOOTING	33
	9.4 KNOWN LIMITATIONS	34
10	**CONCLUSION AND FUTURE SCOPE**	**35**
	10.1 CONCLUSION	35
	10.2 FUTURE SCOPE	35
	REFERENCES	**36**
	APPENDIX A REPOSITORY STRUCTURE AND REPRODUCTION GUIDE	**37**
	APPENDIX B BILL OF MATERIALS	**39**

LIST OF FIGURES

Figure No.	FIGURE CAPTION	PAGE No.
1.1	Embedded system design flow followed in this project	3
4.1	ESP-NOW packet layout	10
5.1	System architecture of HomeGuard	12
6.1	Detector node schematic	15
6.2	Hub schematic	16
6.3	Hub assembled on the breadboard	18
6.4	Detector node inside its enclosure	18
6.5	Reed switch and magnet mounted on the door	19
7.1	Detector firmware flow	21
7.2	Hub firmware flow	22
7.3	Event delivery with acknowledgement, retry and supervision	24
7.4	Alarm state machine (hub)	25
7.5	Phone web page served by the hub	27
8.1	Complete system set up for integration testing	29
8.2	OLED during the exit-delay countdown	31
8.3	System in the ALARM state	32
8.4	Serial monitor logs during validation	32

LIST OF TABLES

Table No.	TABLE TITLE	PAGE No.
1.1	Mapping of report chapters to design-flow stages	3
2.1	Comparison of presence-sensing options	4
2.2	Comparison of wireless links for the detector-to-hub connection	5
3.1	Primary use cases	6
3.2	Functional requirements	6
3.3	Non-functional requirements	7
4.1	Measurable specification	8
4.2	Firmware timing constants	8
4.3	Hub (Board 1) pin assignment	9
4.4	Detector (Board 2) pin assignment	9
4.5	Board MAC addresses used for ESP-NOW peering	10
4.6	Packet fields	10
4.7	Message types	10
5.1	Allocation of functions to nodes and to hardware or software	12
5.2	Key components and selection rationale	13
6.1	Estimated current consumption (typical datasheet values)	17
6.2	Hardware test results	19
7.1	State transition table	25
7.2	LED and buzzer behaviour	26
7.3	Software environment	27
7.4	Firmware source files	27
7.5	Software test results	28
8.1	Response time results (20 door events)	30
8.2	Range test results	30
8.3	Requirement-to-test traceability matrix	30
9.1	Everyday operation	33
9.2	Common faults and remedies	33
A.1	Repository structure	37
B.1	Core bill of materials (approximate Indian retail prices, October 2026)	39

LIST OF SYMBOLS

Symbol	Description
τ	RC time constant (s)
R, R_B	Resistance; transistor base resistance (Ω)
C	Capacitance (F)
I_B, I_C	Transistor base and collector current (A)
h_FE	DC current gain of a bipolar transistor
V_BE	Base–emitter voltage (≈ 0.7 V)
V_CE(sat)	Collector–emitter saturation voltage (V)
V_OH	GPIO output high voltage (3.3 V)
V_F	LED forward voltage (V)
I_LED	LED current (A)
f_SCL	I2C clock frequency (Hz)
t_response	Time from sensor event to siren on (s)

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ACK	Acknowledgement
AP	Access Point
BOM	Bill of Materials
COM / NO	Common / Normally Open switch contacts
CRC	Cyclic Redundancy Check
ESP-NOW	Espressif connection-less peer-to-peer wireless protocol
FR / NFR	Functional / Non-Functional Requirement
FSM	Finite State Machine
GPIO	General-Purpose Input/Output
HT / ST	Hardware Test / Software Test
HW / SW	Hardware / Software
I2C	Inter-Integrated Circuit bus
IDE	Integrated Development Environment
IR	Infrared
ISR	Interrupt Service Routine
LED	Light-Emitting Diode
MAC	Medium Access Control (hardware) address
MCU	Microcontroller Unit
OLED	Organic Light-Emitting Diode
PIN	Personal Identification Number
PIR	Passive Infrared
RAM	Random-Access Memory
RC	Resistor–Capacitor
RTOS	Real-Time Operating System
RTT	Round-Trip Time
SPSC	Single-Producer Single-Consumer (queue)
STA	Wi-Fi Station mode

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Intrusion alarms are one of the most common embedded products found in homes and small offices. A typical system consists of a control panel (hub) and a set of detectors placed at doors, windows and rooms. Older systems wire every detector back to the panel. Wiring is expensive to install, and a single cut cable can silently disable a sensor. Modern commercial kits therefore use battery-powered wireless detectors that report to a hub over a short-range radio link, and the hub supervises every detector so that a missing or damaged detector raises a fault instead of going unnoticed.

This project, HomeGuard, builds a small but complete version of such a system using two ESP32 microcontroller boards: a detector node with a PIR motion sensor, a magnetic door contact and a tamper switch, and a hub with a keypad, an OLED display, a siren, status LEDs and a web page served to a phone. The two boards communicate over ESP-NOW, Espressif's router-less peer-to-peer protocol.

The work was carried out as the term project of the course 25ES604 Embedded System Design. The emphasis of the course, and therefore of this project, is on following the embedded system design flow step by step rather than on the complexity of the application. Every chapter of this report corresponds to one stage of that flow.

1.2 PROBLEM STATEMENT

A low-cost home alarm must solve three problems that a naive design ignores:

- **Silent failures.** If a sensor wire is cut, a battery dies or the detector goes out of radio range, a simple system keeps showing that everything is normal.
- **Tampering.** An intruder who opens or removes a detector should trigger the alarm, even when the system is disarmed.
- **Lost or duplicated messages.** A radio packet can be lost or delivered twice; the alarm must react to every event exactly once.

The problem addressed is therefore: design a two-node wireless intrusion alarm that detects motion and door opening, resists tampering, reports a lost detector, and handles every event reliably, following the complete embedded system design flow.

1.3 OBJECTIVES

1. Capture the functional and non-functional requirements of a wireless intrusion alarm and convert them into a measurable specification.

2. Define a system architecture with a clear hardware/software partition between a detector node and a hub.
3. Design, build and unit-test the detector and hub hardware (sensor interfaces, siren driver, display, keypad).
4. Design, implement and unit-test the firmware: a reliable packet protocol over ESP-NOW, an alarm state machine, detector supervision, PIN security and a phone web page.
5. Integrate both nodes, measure response time and radio range, and validate every requirement through a traceable test.
6. Document operation and maintenance procedures and the known limitations of the system.

1.4 SCOPE AND CONSTRAINTS

- **In scope:** one detector node (motion, door, tamper), one hub, keypad and phone control, local siren and display, supervision of the detector.
- **Out of scope:** multiple detectors, encryption of the radio link, battery operation with deep sleep, SMS or cloud notification. These are listed as future work.
- **Time:** the project had to be completed and demonstrated within about five working days.
- **Cost:** core components cost about ₹2,000 (Appendix B), using off-the-shelf modules on breadboards.
- **Tools:** Arduino IDE 2.x with the ESP32 Arduino core 3.x; no RTOS is used.

1.5 METHODOLOGY

The project follows the embedded system design flow taught in the course (Fig. 1.1). Requirements are fixed first and turned into a specification of measurable numbers. The architecture then partitions the system into hardware and software. Hardware and software are designed, implemented and unit-tested in parallel tracks, each with its own test plan (HT1–HT6 and ST1–ST11). The two tracks are then integrated, validated against the original requirements through a traceability matrix, and finally documented for operation and maintenance.



Fig. 1.1 Embedded system design flow followed in this project

Hardware was brought up one block at a time using small test sketches (lessons L01–L05 in the project repository) before the final firmware was written. This kept every fault local to the block just added and produced a test log for each block.

1.6 ORGANISATION OF THE REPORT

Table 1.1 Mapping of report chapters to design-flow stages

Chapter	Content	Design-flow stage
2	Background and technology survey	Input to requirements
3	Requirements analysis	Stage 1
4	Specification	Stage 2
5	System architecture	Stage 3
6	Hardware design, implementation and testing	Stage 4 (HW track)
7	Software design, implementation and testing	Stage 5 (SW track)
8	System integration and validation	Stages 6 and 7
9	Operation and maintenance	Stage 8
10	Conclusion and future scope	—

CHAPTER 2
BACKGROUND AND TECHNOLOGY SURVEY

2.1 INTRUSION ALARM SYSTEMS

An intrusion alarm detects unauthorised entry and alerts the occupants. Standards such as EN 50131-1 describe the common building blocks: detectors, a control and indicating equipment (the hub), warning devices (sirens), and an interface for setting (arming) and unsetting (disarming) the system [15]. Two properties defined there shape this project: tamper detection, meaning any attempt to open or remove a component raises an alarm, and supervision, meaning the hub must notice when a detector stops communicating.

Commercial wireless kits follow the same pattern: battery-powered detectors report events and periodic heartbeat ("I am alive") messages to a hub, which decides whether to sound the alarm based on its arming state. Entry and exit delays allow the authorised user to leave or enter through the door without setting off the alarm.

2.2 SENSING TECHNOLOGIES

Table 2.1 Comparison of presence-sensing options

Sensor	Principle	Advantages	Limitations
PIR (HC-SR501)	Detects change in infrared from moving warm bodies	Passive, µA-level current, digital output, low cost	Needs warm-up; detects motion only, not a still person
Microwave radar (RCWL-0516)	Doppler shift of reflected microwaves	Wide coverage	Sees through walls, causing false alarms from adjacent rooms
Ultrasonic (HC-SR04)	Echo time of ultrasonic pulses	Measures distance	Continuous pulsing, higher power, narrow beam
Camera	Image processing	Rich information	Cost, power, heavy processing, privacy concerns
Reed switch (MC-38)	Magnet closes contacts	Simple, zero standby power at the contact	Detects door/window opening only

A PIR sensor is the industry-standard room detector, and a magnetic reed contact is the standard door/window detector. This project uses both, plus a mechanical microswitch as the tamper sensor.

2.3 WIRELESS COMMUNICATION OPTIONS

Table 2.2 Comparison of wireless links for the detector-to-hub connection

Option	Needs extra hardware?	Needs router?	Latency	Remarks
Wi-Fi + MQTT	No (ESP32)	Yes	Seconds to connect	Alarm fails if the home Wi-Fi is down
Bluetooth LE	No (ESP32)	No	Tens of ms	More complex pairing, shorter range
LoRa	Yes (radio module)	No	Hundreds of ms	Very long range, low data rate
Zigbee	Yes (radio module)	No (coordinator)	Tens of ms	Mesh networking, extra stack
ESP-NOW	No (ESP32)	No	A few ms	Selected: built-in, peer-to-peer by MAC address

ESP-NOW is a connection-less protocol from Espressif that sends short frames (up to 250 bytes) directly between ESP32 devices using their MAC addresses, without joining a Wi-Fi network [4]. Both peers must use the same Wi-Fi channel. Its built-in acknowledgement confirms only that the receiving radio got the frame, not that the receiving application processed it; this gap is closed in Chapter 7 with an application-level protocol.

2.4 EMBEDDED DESIGN CONCEPTS APPLIED

- **Fail-safe input design:** switches pull the input to ground, so an open circuit (cut wire) reads the same as an alarm condition.
- **Contact debouncing:** hardware RC filtering combined with a software stability window [14].
- **Interrupt-driven input with a non-blocking super-loop:** interrupts only record that an event happened; all decisions are taken in the main loop.
- **Finite state machine (FSM):** the alarm logic is a small set of states with explicit transitions, which makes it easy to test exhaustively [1], [2].
- **Reliable messaging:** sequence numbers, checksums (CRC), acknowledgements, retries and duplicate filtering [11].

CHAPTER 3
REQUIREMENTS ANALYSIS

The first stage of the design flow answers what the system must do, without deciding how. Requirements were collected from the problem statement, from the behaviour of commercial wireless alarms and from the intrusion-alarm concepts in Chapter 2. Each requirement has an identifier so that it can be traced through specification, design and testing.

3.1 STAKEHOLDERS AND USE CASES

Table 3.1 Primary use cases

Use case	Actor	Description
UC1 Arm	Occupant	Enter PIN and arm the system; leave within the exit delay
UC2 Enter and disarm	Occupant	Open the door and enter the PIN within the entry delay
UC3 Intrusion	Intruder	Motion in the room or door opened while armed raises the alarm
UC4 Tamper	Intruder	Opening the detector enclosure raises the alarm in any state
UC5 Sensor lost	System	Detector loses power or goes out of range; hub shows a fault
UC6 Remote control	Occupant	Check status and arm/disarm from a phone on the hub's Wi-Fi

3.2 FUNCTIONAL REQUIREMENTS

Table 3.2 Functional requirements

ID	Requirement
FR1	The system shall detect motion in the protected room.
FR2	The system shall detect the door opening and closing.
FR3	The system shall detect tampering with the detector enclosure in every state, including when disarmed.
FR4	The user shall be able to arm and disarm the system with a 4-digit PIN from a keypad and from a phone.
FR5	The system shall provide an exit delay of 30 s after arming and an entry delay of 15 s after the door opens.
FR6	The system shall sound a siren on alarm and show its status on LEDs, a display and the phone page.
FR7	The system shall report a detector that has stopped communicating.
FR8	The system shall lock out PIN entry after 3 consecutive wrong PINs.

3.3 NON-FUNCTIONAL REQUIREMENTS

Table 3.3 Non-functional requirements

ID	Requirement	Rationale
NFR1	Time from a sensor event to siren on shall be under 1 s.	An alarm must react before an intruder can reach the hub.
NFR2	No event shall be lost; every event shall be acknowledged.	A missed door or motion event defeats the alarm.
NFR3	A cut wire or a dead detector shall never fail silently.	Fail-safe behaviour is the core of a security product.
NFR4	Indoor radio range shall be at least 10 m.	Covers a typical flat from one hub location.
NFR5	The system shall work without internet or a Wi-Fi router.	The alarm must keep working during a network outage.

3.4 DESIGN CONSTRAINTS

- Two ESP32 DevKit V1 boards (ESP32-D0WD-V3, rev 3.1, 4 MB flash) are used; no additional radio module.
- All parts must be low-cost, readily available modules that can be assembled on breadboards.
- The firmware is written in C++ with the Arduino framework; a bare-metal super-loop is preferred over an RTOS because the number of tasks is small and the deadlines are in the millisecond range.
- The complete design, build, test and documentation must fit into about five working days.

CHAPTER 4
SPECIFICATION

The specification stage turns every requirement into numbers and interfaces that can be designed against and tested. Table 4.1 lists the measurable parameters together with the requirement each one traces back to.

4.1 SYSTEM SPECIFICATION

Table 4.1 Measurable specification

Parameter	Specification	Traces to
Radio link	ESP-NOW, 2.4 GHz, Wi-Fi channel 1, peer-to-peer by MAC address, no router	NFR4, NFR5
Packet	12-byte packed structure with node ID, type, sequence number, battery, uptime and CRC-16/CCITT	NFR2
Reliability	Hub ACK within 100 ms; up to 3 transmissions; duplicates filtered by sequence number	NFR2
Supervision	Heartbeat every 5 s; detector declared OFFLINE after 15 s without any packet	FR7, NFR3
Switch inputs	Switch to GND, 10 kΩ pull-up, 100 nF to GND (τ = 1 ms), 50 ms software debounce	FR2, FR3, NFR3
PIR input	Digital 3.3 V output on an interrupt pin; 30 s warm-up ignored	FR1
Alarm timing	Exit delay 30 s; entry delay 15 s; siren limited to 3 min per alarm	FR5, FR6
Security	4-digit PIN; 3 wrong attempts cause a 60 s lockout of keypad and phone	FR4, FR8
User interface	4×4 keypad, 128×64 OLED (I2C, 0x3C), 4 LEDs, buzzer, web page at http://192.168.4.1	FR4, FR6
Response time	Sensor event to siren on < 1 s	NFR1

4.2 TIMING CONSTANTS

Table 4.2 Firmware timing constants

Constant	Value	Node	Purpose
DEBOUNCE_MS	50 ms	Detector	Minimum stable time before a switch change is accepted
PIR_WARMUP_MS	30 000 ms	Detector	PIR output ignored after power-up
HEARTBEAT_MS	5 000 ms	Detector	Interval between heartbeat packets

Constant	Value	Node	Purpose
ACK_TIMEOUT_MS	100 ms	Detector	Wait for ACK before retransmitting
MAX_RETRIES	3	Detector	Total transmissions per event
EXIT_DELAY_MS	30 000 ms	Hub	Time to leave after arming
ENTRY_DELAY_MS	15 000 ms	Hub	Time to disarm after the door opens
OFFLINE_MS	15 000 ms	Hub	Silence that marks the detector offline (3 missed heartbeats)
LOCKOUT_MS	60 000 ms	Hub	Keypad/phone lockout after 3 wrong PINs
SIREN_MAX_MS	180 000 ms	Hub	Maximum siren time per alarm

4.3 INTERFACE SPECIFICATION

4.3.1 Pin assignment

Table 4.3 Hub (Board 1) pin assignment

Signal	ESP32 pin	Direction	Notes
Siren driver	GPIO25	Output	Through 1 kΩ to BC547 base
LED READY / ARMED / ALARM / FAULT	GPIO26 / 27 / 14 / 13	Output	330 Ω series resistor each
OLED SDA / SCL	GPIO21 / GPIO22	I2C	SSD1306 at address 0x3C, 3.3 V
Keypad rows R1–R4	GPIO19, 18, 23, 17	Output (scan)	Keypad pins 1–4
Keypad columns C1–C4	GPIO16, 4, 32, 33	Input (pull-up)	Keypad pins 5–8
Power	VIN (5 V USB), 3V3, GND	—	Buzzer from 5 V, logic from 3.3 V

Table 4.4 Detector (Board 2) pin assignment

Signal	ESP32 pin	Direction	Notes
PIR OUT	GPIO27	Input, interrupt on rising edge	HC-SR501 powered from VIN (5 V)
Door reed switch	GPIO26	Input	10 kΩ pull-up + 100 nF; LOW = door closed
Tamper microswitch (COM–NO)	GPIO25	Input	10 kΩ pull-up + 100 nF; LOW = lid closed
Transmit indicator	GPIO2	Output	On-board LED flashes 30 ms per packet

4.3.2 Board identities

Table 4.5 Board MAC addresses used for ESP-NOW peering

Role	Label	Station MAC address
Hub	Board 1	FC:E8:C0:75:C1:94
Detector	Board 2	D0:EF:76:33:EE:44

Each board stores the MAC address of its peer. A different pair of boards can be used by reading their MAC addresses with the L01 board-check sketch and updating `HUB_MAC` in the detector sketch and `DET_MAC` in the hub sketch.

4.4 MESSAGE SPECIFICATION

All communication uses one fixed 12-byte packet (Fig. 4.1, Table 4.6). The structure is declared with `__attribute__((packed))` so that both boards see exactly the same byte layout without compiler padding.

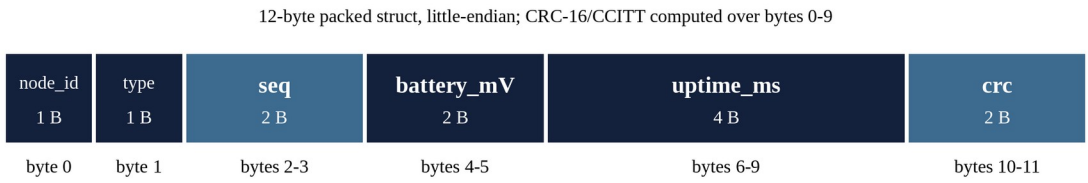


Fig. 4.1 ESP-NOW packet layout

Table 4.6 Packet fields

Offset	Field	Size	Meaning
0	node_id	1 B	Sender: 0 = hub, 1 = detector
1	type	1 B	Message type (Table 4.7)
2–3	seq	2 B	Sequence number; incremented per new event, unchanged on retry, echoed in the ACK
4–5	battery_mV	2 B	Reserved for battery voltage (0 on USB power)
6–9	uptime_ms	4 B	Sender millis(); a sudden drop reveals a reboot
10–11	crc	2 B	CRC-16/CCITT (polynomial 0x1021, initial value 0xFFFF) over bytes 0–9

Table 4.7 Message types

Value	Name	Sent by	Meaning
0	HEARTBEAT	Detector	Periodic "alive" message every 5 s

Value	Name	Sent by	Meaning
1	MOTION	Detector	PIR output rose (after warm-up)
2	DOOR_OPEN	Detector	Reed switch opened
3	DOOR_CLOSED	Detector	Reed switch closed
4	TAMPER	Detector	Enclosure lid opened
5	ACK	Hub	Event received and handled (same seq)
6	TAMPER_CLEAR	Detector	Enclosure lid closed again

Example: a MOTION event from the detector with sequence number 41 and uptime 123 456 ms is transmitted as the bytes `01 01 29 00 00 00 40 E2 01 00 C2 D4`, where `0xD4C2` is the CRC of the first ten bytes (multi-byte fields are little-endian).

CHAPTER 5
SYSTEM ARCHITECTURE

The architecture stage decides how the system is split into hardware and software blocks and how those blocks communicate. Fig. 5.1 shows the result: two ESP32 nodes joined by a single ESP-NOW link.

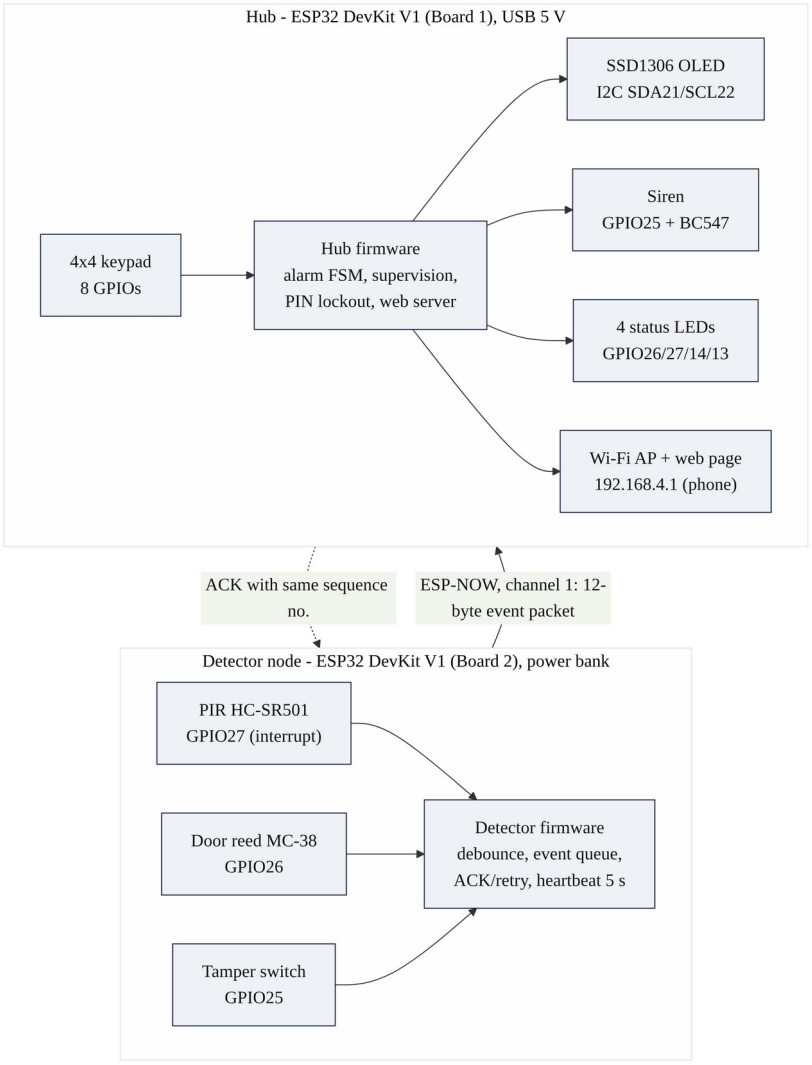


Fig. 5.1 System architecture of HomeGuard

5.1 HARDWARE/SOFTWARE PARTITIONING

Table 5.1 Allocation of functions to nodes and to hardware or software

Function	Node	Hardware	Software
Motion sensing	Detector	PIR module (analogue front end on module)	Interrupt flag, warm-up gate
Door and tamper sensing	Detector	Switches, pull-ups, RC filter	50 ms debounce, change detection

Function	Node	Hardware	Software
Event delivery	Both	ESP32 2.4 GHz radio	Packet, CRC, ACK, retry, duplicate filter
Supervision	Both	—	Heartbeat (detector), 15 s timeout (hub)
Alarm decision	Hub	—	Finite state machine
User input	Hub	4×4 keypad matrix, Wi-Fi AP	Key scanning, PIN check, lockout, web form
Alarm output	Hub	Transistor siren driver, LEDs, OLED	Siren pattern, LED logic, display refresh

5.2 WHY TWO MICROCONTROLLERS

The detector sits at the door while the hub sits inside the home where the user can reach the keypad. Splitting the system into two nodes mirrors the real product and makes the wireless link, its reliability and its supervision the central engineering problem of the project. The detector is deliberately simple: it only senses and reports. Every decision, including whether an event should sound the siren, is taken by the hub. This keeps the detector firmware small and makes the hub the single place where the alarm policy is implemented and tested.

5.3 WHY NO RTOS

Each node has only a handful of activities (read inputs, send or receive packets, update outputs, serve a web page) and all deadlines are in the millisecond range. A non-blocking super-loop in which interrupts and radio callbacks only copy data into a queue meets these deadlines, uses less memory and is easier to reason about and test than a multi-task RTOS design. The ESP32 Arduino core itself runs on FreeRTOS, but the application does not create additional tasks.

5.4 COMPONENT SELECTION

Table 5.2 Key components and selection rationale

Component	Selected part	Reason
Microcontroller	ESP32 DevKit V1 (×2)	Built-in Wi-Fi radio supports ESP-NOW and a soft access point; dual core, 3.3 V I/O, low cost
Motion sensor	HC-SR501 PIR	Industry-standard principle, µA standby current, 3.3 V digital output, adjustable range and hold time
Door sensor	MC-38 reed switch	Wired magnetic contact, no power needed at the contact

Component	Selected part	Reason
Tamper sensor	Lever microswitch (NO)	Pressed by the enclosure lid; opens when the lid is removed
Display	SSD1306 0.96" OLED, I2C	Only two signal wires, readable at any angle, 3.3 V
Keypad	4×4 membrane keypad	16 keys from 8 GPIOs; digits plus A (arm), # (disarm), * (clear)
Siren	5 V active buzzer + BC547	Active buzzer needs only on/off; transistor provides 5 V switching

CHAPTER 6
HARDWARE DESIGN, IMPLEMENTATION AND TESTING

This chapter covers the hardware track of the design flow: circuit design with calculations, breadboard implementation and the hardware tests HT1–HT6.

6.1 DETECTOR NODE DESIGN

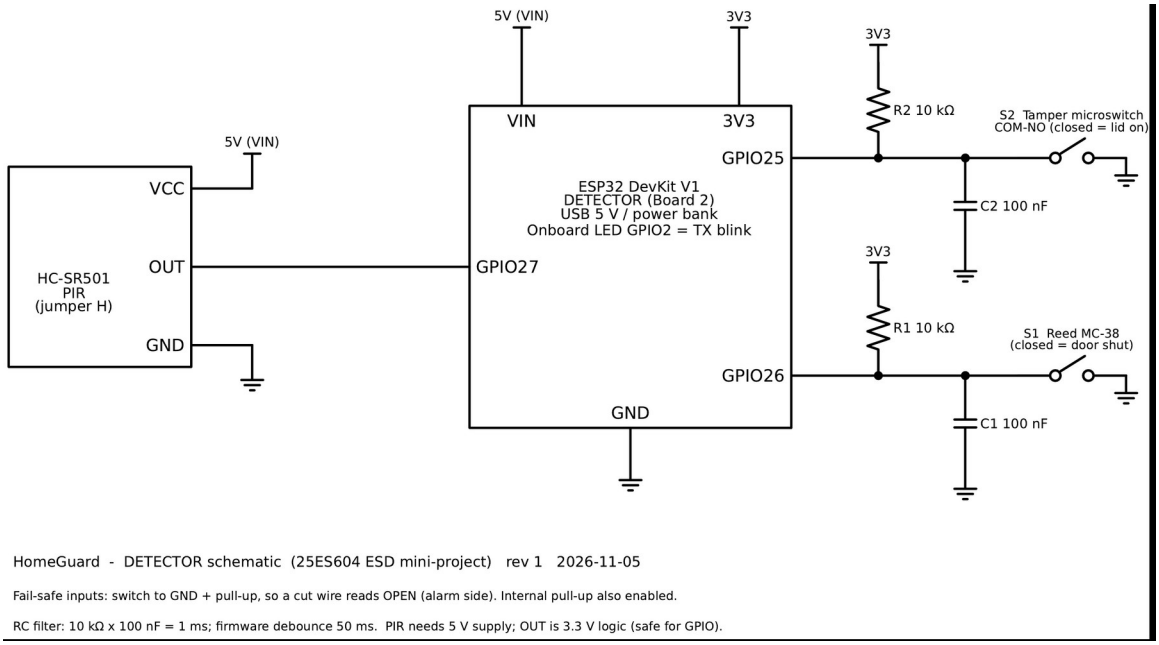


Fig. 6.1 Detector node schematic

6.1.1 Fail-safe switch inputs

Both the reed switch and the tamper microswitch connect their GPIO pin to ground when closed. A 10 kΩ resistor pulls the pin up to 3.3 V when the switch is open. With this arrangement the normal, secure state (door shut, lid on) reads LOW, and every abnormal state (door open, lid removed, or a cut wire) reads HIGH. A broken connection therefore produces an event instead of silently disabling the sensor, which satisfies NFR3. The ESP32's internal pull-up is enabled as well as a backup.

The pull-up value is a trade-off. With the switches closed most of the time, the resistor draws a continuous current I = 3.3 V / R. A 1 kΩ resistor would waste 3.3 mA per switch; 10 kΩ reduces this to 0.33 mA while still holding the line firmly against noise. A much larger value (100 kΩ) would make long wires susceptible to interference.

6.1.2 Debouncing

Mechanical contacts bounce for several milliseconds when they change state. Two layers handle this. A 100 nF capacitor from the pin to ground forms an RC low-pass filter with the pull-up: $\tau = R \times C = 10\ k\Omega \times 100\ nF = 1$ ms, which suppresses short spikes

and noise picked up on the wire. Bounce that lasts longer than the filter time is removed in software: a change is accepted only after the input has been stable for 50 ms. During testing the tamper microswitch produced up to 31 raw edges for a single press, yet exactly one event was reported (Section 6.5).

6.1.3 PIR sensor interface

The HC-SR501 contains a pyroelectric sensor behind a Fresnel lens and a BISS0001 signal-processing IC. It requires a 4.5–20 V supply and is powered from the 5 V VIN pin, while its on-board regulator produces a 3.3 V digital output that connects directly to GPIO27 without level shifting. The trigger jumper is set to H (repeatable trigger) and the time potentiometer is recommended to be at its minimum, since the firmware reacts to the rising edge only. The output is clean digital logic, so no debounce is required; the firmware ignores it for 30 s after power-up while the sensor stabilises.

6.2 HUB DESIGN

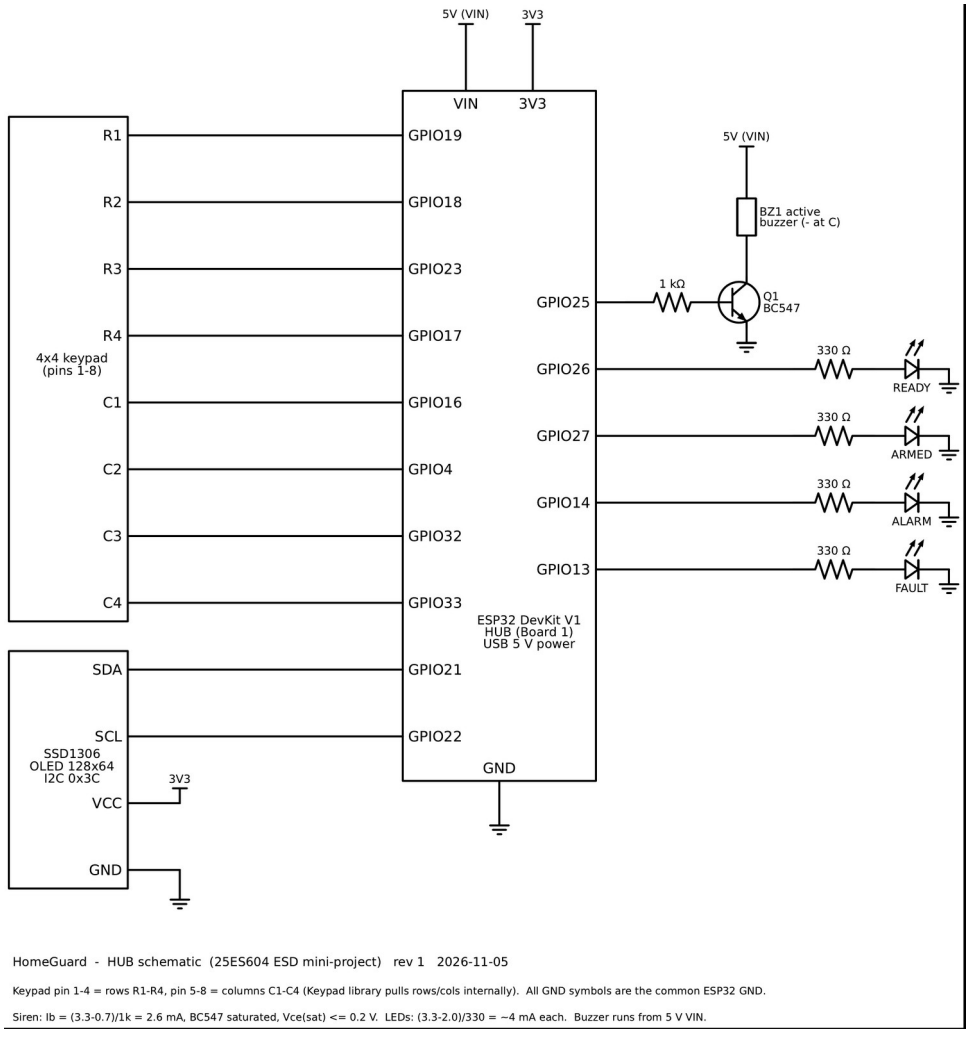


Fig. 6.2 Hub schematic

6.2.1 Siren driver

A GPIO pin can supply only a few milliamperes at 3.3 V, while the buzzer is specified for 5 V and draws more current than a pin should supply. A BC547 NPN transistor is therefore used as a low-side switch: the buzzer is connected between the 5 V rail and the collector, and the emitter goes to ground. The base is driven from GPIO25 through a 1 kΩ resistor.

Base current: $I_B = (V_{OH} - V_{BE}) / R_B = (3.3 - 0.7)\ V / 1\ k\Omega = 2.6\ mA$. With a current gain h_{FE} of at least 100, the transistor could conduct about 260 mA, far more than the buzzer needs, so it is driven into saturation and $V_{CE(sat)} \leq 0.2$ V. The buzzer therefore receives almost the full 5 V. The base resistor limits the GPIO current; without it the base-emitter junction would act almost as a short circuit. An active buzzer is used, so no tone generation is needed; a flyback diode (1N4148) is recommended if a magnetic buzzer is substituted.

6.2.2 Status LEDs

Each LED is driven directly from a GPIO through a 330 Ω resistor: $I_{LED} = (3.3 - 2.0)\ V / 330\ \Omega \approx 4\ mA$, which gives clearly visible indication while staying well within the pin's current capability.

6.2.3 Keypad and display

The 4×4 membrane keypad is a matrix: rows are driven one at a time and the columns are read, so 16 keys need only 8 GPIOs. The SSD1306 OLED uses I2C, a two-wire bus (SDA, SCL) with open-drain lines and pull-up resistors on the module. The ESP32 is the bus master and the display is a slave at the 7-bit address 0x3C; the display is powered from 3.3 V, so no level shifting is needed.

6.3 POWER BUDGET

Table 6.1 Estimated current consumption (typical datasheet values)

Load	Detector	Hub
ESP32 with radio receiving continuously	≈ 100 mA	≈ 100 mA
PIR module (idle)	≈ 0.065 mA	—
Pull-ups with switches closed	0.66 mA	—
OLED (typical text screen)	—	≈ 10–20 mA
LEDs (one or two on)	—	4–8 mA
Buzzer (only during alarm)	—	≈ 30 mA
Estimated total	≈ 100 mA	≈ 115–160 mA

Both nodes run from 5 V USB: the hub from a USB charger or laptop and the detector from a power bank. The detector keeps its radio on continuously, so a 10 000 mAh power bank would last on the order of three to four days; measured current: [to be filled]. Battery operation with deep sleep is identified as future work.

6.4 HARDWARE IMPLEMENTATION

Each node was assembled on a breadboard. The hub uses an 830-point breadboard with the keypad and OLED connected by jumper leads; the detector circuit is built on a half breadboard sized to fit inside a small ABS enclosure, with the PIR dome looking out through the lid and the tamper lever pressed by the closed lid. Continuity was checked with a multimeter before power was applied to each new block.

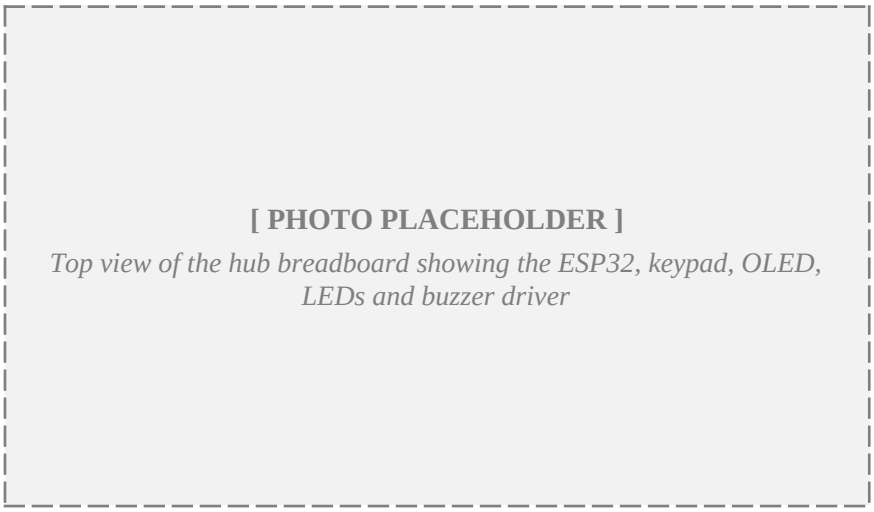


Fig. 6.3 Hub assembled on the breadboard

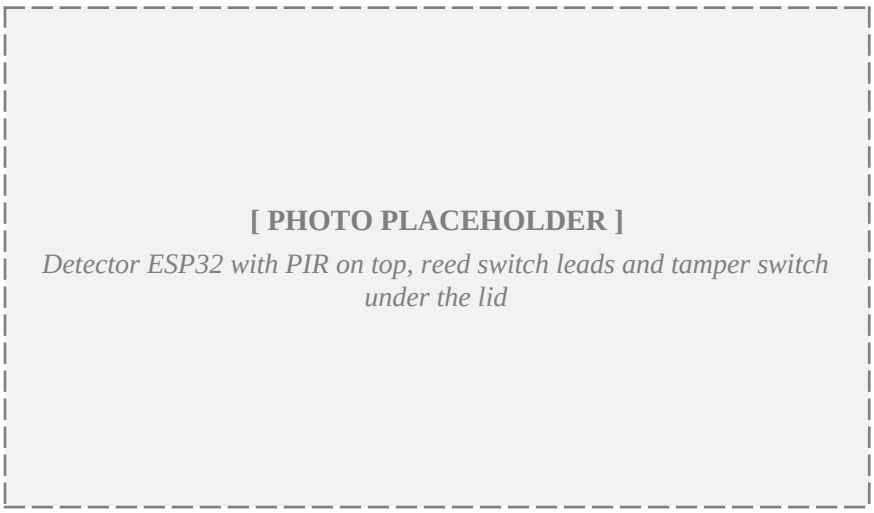


Fig. 6.4 Detector node inside its enclosure

[PHOTO PLACEHOLDER]

Reed switch on the frame and magnet on the door edge, gap 10 mm or less

Fig. 6.5 Reed switch and magnet mounted on the door

6.5 HARDWARE TESTING

Every hardware block was tested on its own with a dedicated test sketch before the final firmware was loaded. Table 6.3 lists the hardware tests, their pass criteria and the results; the serial logs are stored in the repository under `docs/06_testing/logs/`.

Table 6.2 Hardware test results

ID	Block	Pass criterion	Measured / observed	Result
HT1	Power rails	5.0 ± 0.25 V; 3.3 ± 0.1 V	[to be filled]	[to be filled]
HT2	PIR sensor	Detects a walk-past; OUT ≈ 3.3 V	5 of 5 walk-pasts detected; output HIGH for 1.06–2.55 s per event; range at 1/2/4 m: [to be filled]	Pass (detection)
HT3	Door and tamper inputs	One event per action; 0 V closed, 3.3 V open	One debounced event per action. Reed: 1 raw edge per action; tamper: 1–31 raw edges per action. Voltages: [to be filled]	Pass (logic)
HT4	Siren driver	Buzzer sounds; V_CE ≤ 0.2 V when on	[to be filled]	[to be filled]
HT5	LEDs, keypad, OLED	4 LEDs; 16 keys; OLED found at 0x3C	All 4 LEDs pass. Keypad and OLED: [to be filled]	Pass (LEDs)
HT6	Radio link	≥ 99 of 100 pings answered	0 lost in 452 consecutive pings; round-trip time typically 2.4–3.0 ms, maximum 28.8 ms (same desk). 10 m: [to be filled]	Pass (desk)

The L03 log illustrates why both debounce layers are needed: the reed switch changed cleanly with a single raw edge, while the lever microswitch bounced heavily, producing

between 3 and 31 raw transitions per action. In every case the debounced output reported exactly one event. In the PIR test (L04), five consecutive walk-pasts produced five MOTION events with the output held HIGH for between 1 055 ms and 2 546 ms.

CHAPTER 7
SOFTWARE DESIGN, IMPLEMENTATION AND TESTING

This chapter covers the software track: the firmware structure, the communication protocol, the alarm state machine, implementation details and the software tests ST1–ST11.

7.1 FIRMWARE STRUCTURE

Both nodes use the same structure: a non-blocking super-loop with no `delay()` calls. Interrupt service routines and the ESP-NOW receive callback do the minimum possible work (copy data, set a flag) and return immediately; all decisions are made in `loop()`. Fig. 7.1 and Fig. 7.2 show the flow of each node.

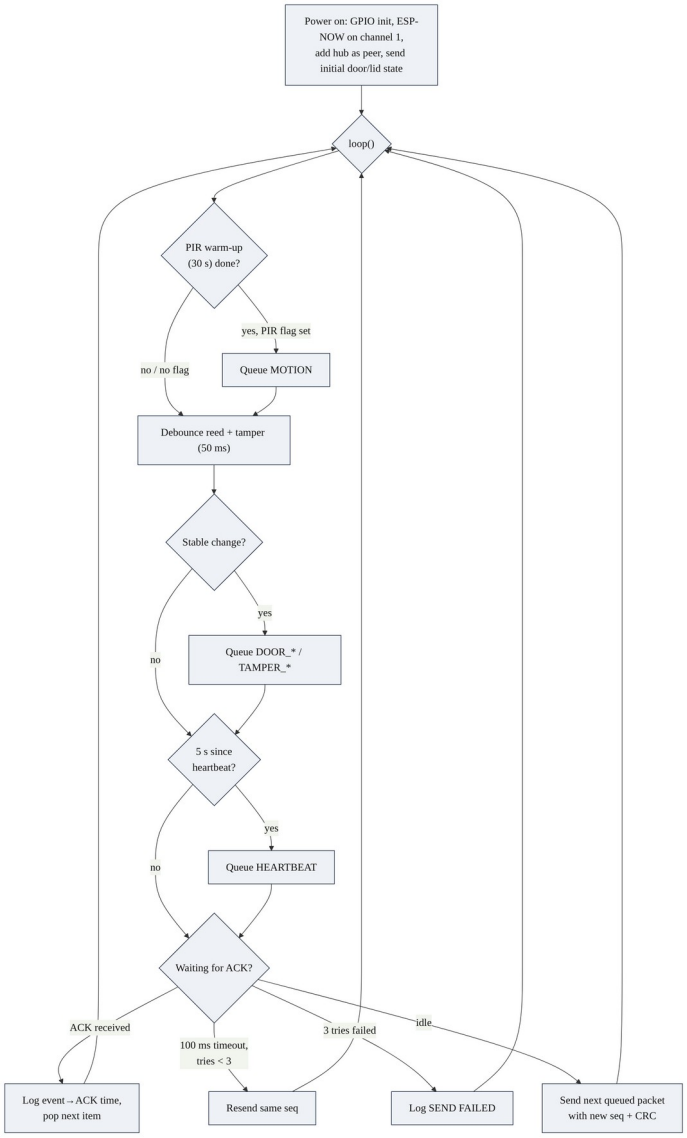


Fig. 7.1 Detector firmware flow

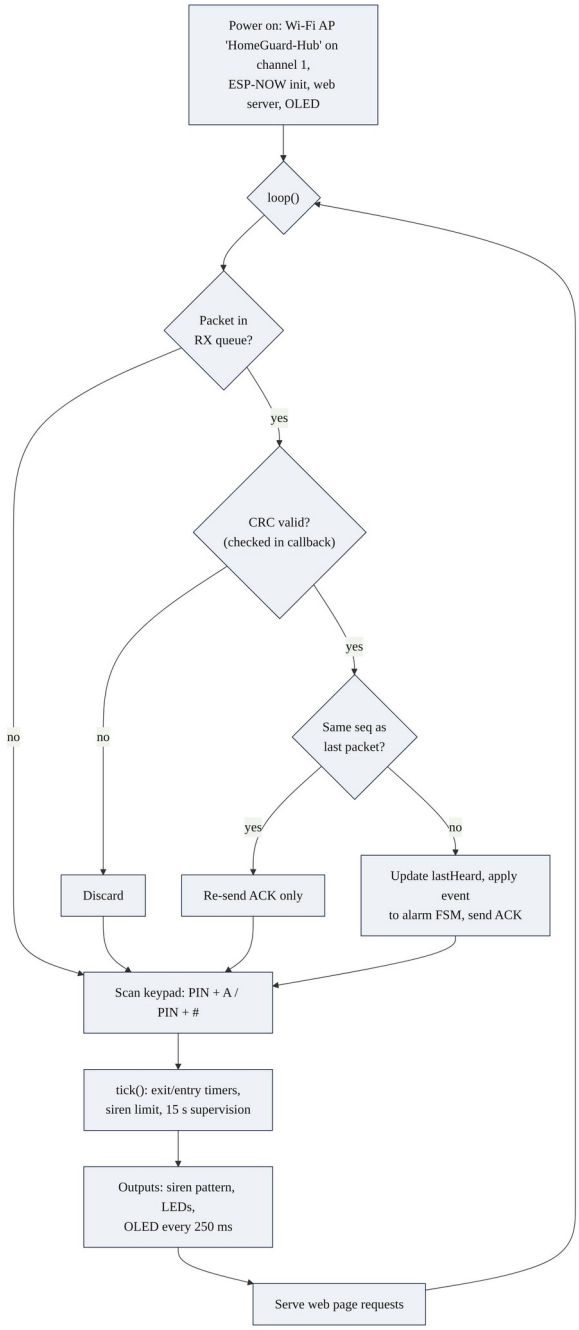


Fig. 7.2 Hub firmware flow

7.1.1 Interrupt and callback design

On the detector, the PIR output triggers an interrupt on its rising edge. The ISR is marked `IRAM_ATTR` so that it executes from internal RAM, and it only records a flag and a timestamp. On the hub, the ESP-NOW receive callback runs in the Wi-Fi task; it validates the length and CRC, copies the packet into an 8-entry single-producer/single-consumer ring buffer and returns.

Listing 7.1 Minimal interrupt and receive-callback handlers

```
void IRAM_ATTR onPirRise() {        // detector: PIR interrupt
  pirRise = true;                     // only set a flag ...
  pirRiseMs = millis();               // ... and a timestamp
}

void onRecv(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
  if (len != (int)sizeof(Packet)) return;         // hub: radio callback
  Packet p;  memcpy(&p, data, sizeof(Packet));
  if (!packetValid(p)) { rxBadCrc++; return; }    // CRC check
  int next = (rxTail + 1) % RXQ;
  if (next == rxHead) { rxDropped++; return; }    // queue full
  rxq[rxTail] = p;  rxTail = next;                // hand over to loop()
}
```

7.2 COMMUNICATION PROTOCOL

ESP-NOW's link-layer acknowledgement only shows that the hub's radio received a frame. To guarantee that every event is handled exactly once (NFR2), an application-level protocol is added on top (Fig. 7.3):

- Each new event gets the next **sequence number**; the packet is sealed with a **CRC-16**.
- The hub acts on the event first (for example, switches the siren on) and only then sends an **ACK carrying the same sequence number**. An ACK therefore means "handled".
- If no matching ACK arrives within **100 ms**, the detector retransmits the same packet, up to **3** transmissions in total, and logs `SEND FAILED` if all fail.
- If the hub receives a sequence number equal to the last one it handled, it treats the packet as a **duplicate**: it re-sends the ACK but does not act again.
- A packet whose CRC does not match is discarded.
- The detector sends a **heartbeat** every 5 s; if the hub hears nothing for 15 s it declares the detector **OFFLINE** (FR7).

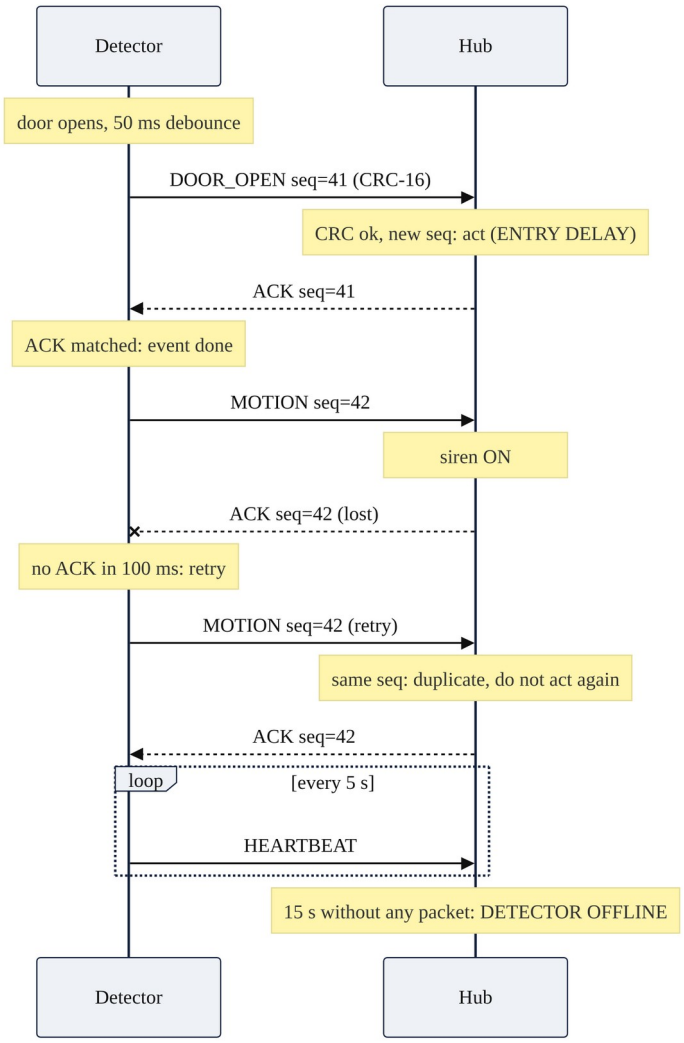


Fig. 7.3 Event delivery with acknowledgement, retry and supervision

Listing 7.2 CRC-16/CCITT used to seal and verify every packet

```
inline uint16_t crc16_ccitt(const uint8_t *d, size_t n) {
  uint16_t crc = 0xFFFF;
  while (n--) {
    crc ^= (uint16_t)(*d++) << 8;
    for (int i = 0; i < 8; i++)
      crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
  }
  return crc;
}
```

7.3 ALARM STATE MACHINE

The alarm policy is implemented entirely on the hub as a finite state machine with five states (Fig. 7.4, Table 7.1). The normal path runs DISARMED → EXIT DELAY → ARMED → ENTRY DELAY → ALARM. Motion while armed skips the entry delay, tamper reaches ALARM from any state, and a valid PIN followed by # returns any state

to DISARMED. Alarms latch: closing the detector lid or the door does not silence the siren; only a valid PIN does.

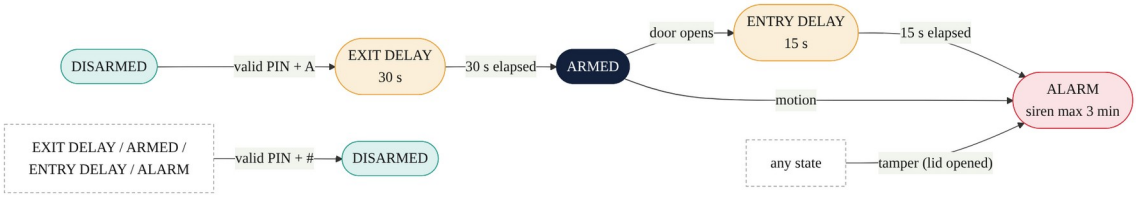


Fig. 7.4 Alarm state machine (hub)

Table 7.1 State transition table

Current state	Event / condition	Next state	Action
DISARMED	Valid PIN + A, lid closed, detector online	EXIT DELAY	Start 30 s countdown, chirp each second
DISARMED	Valid PIN + A, lid open or detector offline	DISARMED	Log "Cannot arm"
EXIT DELAY	30 s elapsed	ARMED	ARMED LED steady
ARMED	DOOR_OPEN	ENTRY DELAY	Start 15 s countdown, chirp each second
ARMED	MOTION	ALARM	Siren on immediately
ENTRY DELAY	15 s elapsed	ALARM	Siren on
Any state	TAMPER	ALARM	Siren on (24-hour zone)
EXIT DELAY / ARMED / ENTRY DELAY / ALARM	Valid PIN + #	DISARMED	Siren off
ALARM	Siren on for 3 min	ALARM	Siren stops; state remains ALARM until a valid PIN
Any state	Wrong PIN (3rd consecutive)	unchanged	Keypad and phone locked for 60 s

Listing 7.3 State change with immediate siren control

```
void setState(AlarmState s, const char *reason) {
  state = s;
  stateSinceMs = millis();
  sirenTimedOut = false;
  if (s == ST_ALARM)    digitalWrite(PIN_SIREN, HIGH);  // siren on before
the ACK
  if (s == ST_DISARMED) digitalWrite(PIN_SIREN, LOW);
  addLog("STATE -> %s (%s)", STATE_NAMES[s], reason);
}
```

7.4 USER INTERFACE SOFTWARE

7.4.1 Keypad and PIN handling

Digits are collected into a 4-character buffer; `*` clears it, `A` submits it as an arm request and # as a disarm request. The same `handlePin()` function serves the keypad and the phone, so the PIN check, the wrong-PIN counter and the 60 s lockout apply to both. Arming is refused when the detector lid is open or the detector is offline.

7.4.2 OLED display

The display shows the state in large text, the exit or entry countdown (or the last event), the detector status and the PIN entry as asterisks or the lockout timer. Drawing happens in a 1 024-byte frame buffer in RAM; `display()` then sends the whole buffer over I2C. At 400 kHz a full transfer of about 9 200 bits takes roughly 23 ms and blocks the loop, so the screen is refreshed every 250 ms rather than on every loop pass. This keeps about 90 % of the time free for the radio and keypad.

7.4.3 Outputs and indications

Table 7.2 LED and buzzer behaviour

Output	Behaviour
READY LED	On when DISARMED, detector online and lid closed
ARMED LED	Steady when ARMED; blinking during exit and entry delays
ALARM LED	On in ALARM
FAULT LED	On when the detector is offline or the lid is open; blinking during PIN lockout
Buzzer	Continuous in ALARM (max 3 min); 40 ms chirp every second during delays; 40 ms chirp every 10 s while the detector is offline

7.4.4 Phone web page

The hub runs a Wi-Fi access point HomeGuard-Hub on channel 1, the same channel used for ESP-NOW, so a single radio serves both functions. A phone or laptop that joins this network opens `http://192.168.4.1` to see the state, detector status, door and lid status and the 20 most recent events. A form submits the PIN with an Arm or Disarm action to `/cmd`, which calls `handlePin()` and redirects back to the page. The page refreshes every 2 s, but skips the refresh while the PIN box is in use so that typing is not interrupted. Because the access point needs no router or internet, the page works during a network outage (NFR5).

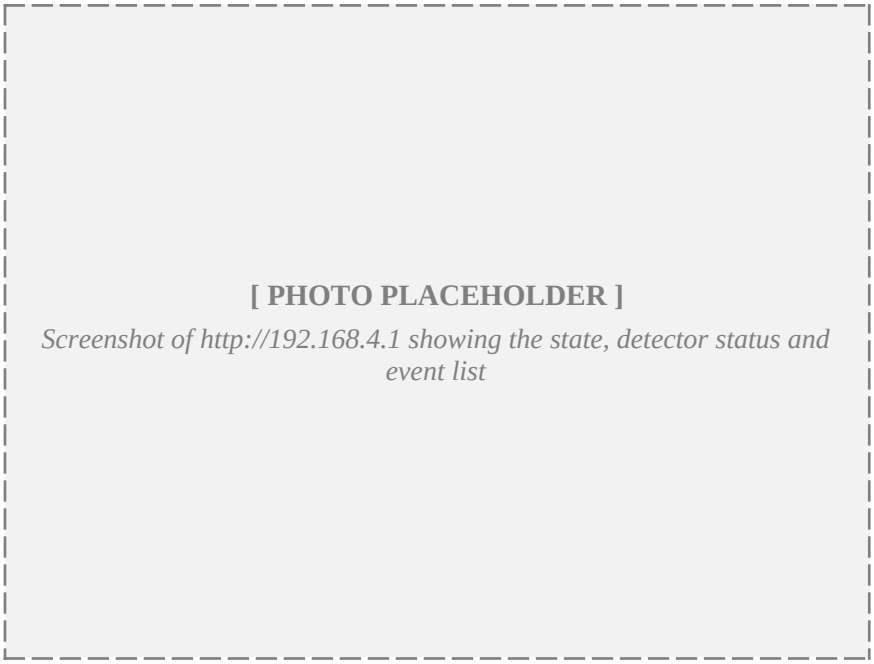


Fig. 7.5 Phone web page served by the hub

7.5 IMPLEMENTATION

Table 7.3 Software environment

Item	Version / detail
IDE	Arduino IDE 2.x
Board package	ESP32 Arduino core 3.x (Espressif); board: ESP32 Dev Module
Libraries	Adafruit SSD1306, Adafruit GFX, Keypad (Mark Stanley, Alexander Brevig); WiFi, WebServer and esp_now from the core
Language	C++ (Arduino framework), no RTOS tasks created by the application
Version control	Git, GitHub repository esp32-wireless-intrusion-alarm

Table 7.4 Firmware source files

File	Node	Purpose
firmware/common/packet.h	Both	Master copy of the packet, message types and CRC
firmware/detector_node/ detector_node.ino	Detector	Inputs, debounce, event queue, ACK/retry, heartbeat
firmware/detector_node/ debounced_input.h	Detector	Debounced switch input helper
firmware/hub/hub.ino	Hub	Alarm FSM, supervision, keypad, OLED, outputs, web server
firmware/test_sketches/L01–L05	Either	Board check, ESP-NOW ping, input, PIR and hub I/O unit tests

Two implementation issues were met and fixed. First, the Arduino IDE generates function prototypes and inserts them before the first function in a sketch; when a structure type was declared after the first function, compilation failed with "'DebouncedInput' was not declared in this scope". The fix was to move all types into a header (`debounced_input.h`) included at the top. Second, the ESP-NOW receive callback signature differs between ESP32 core 2.x and 3.x; the code selects the correct signature with `ESP_ARDUINO_VERSION_MAJOR`.

7.6 SOFTWARE TESTING

Software tests were run on the real boards with the serial monitors of both nodes open. Table 7.6 lists the tests and their results.

Table 7.5 Software test results

ID	Test	Pass criterion	Result
ST1	Event delivery: open/close the door 5 times	10 of 10 events ACKed on the first try; event → ACK < 50 ms	[to be filled]
ST2	Arm with exit delay (PIN + A)	30 s countdown with chirps, then ARMED	[to be filled]
ST3	Entry and disarm within 15 s	ENTRY DELAY then DISARMED, no siren	[to be filled]
ST4	Entry delay timeout	ALARM after 15 s, siren on	[to be filled]
ST5	Motion while armed	ALARM immediately	[to be filled]
ST6	Tamper while disarmed	ALARM; valid PIN silences it	[to be filled]
ST7	Detector power removed	DETECTOR OFFLINE within 15 s; ONLINE on return	[to be filled]
ST8	Three wrong PINs	Keypad and phone locked for 60 s	[to be filled]
ST9	Arm with lid open	Arming refused	[to be filled]
ST10	Phone page	Status updates ≤ 2 s; arm and disarm work	Pass (laptop browser)
ST11	Retry and duplicate filter	Retried event handled once	[to be filled]

CHAPTER 8
SYSTEM INTEGRATION AND VALIDATION

8.1 INTEGRATION PROCEDURE

1. **Pairing:** each sketch contains the peer's MAC address; both nodes fix the radio to Wi-Fi channel 1. The hub's access point is also on channel 1 so that ESP-NOW and the web page share the radio.
2. **Mounting:** the detector enclosure is placed above the door with the PIR on top facing the room, the reed switch on the frame with the magnet on the door, and the tamper switch under the lid.
3. **Going wireless:** the detector is powered from a USB power bank and the hub from USB, so the only connection between them is the radio link.
4. **End-to-end check:** the hub log must show `Detector ONLINE` followed by the initial door and lid states, and the detector log must show an ACK for each event.



Fig. 8.1 Complete system set up for integration testing

8.2 RESPONSE TIME MEASUREMENT

The detector timestamps each event when it is queued and prints the time until the matching ACK arrives (`event->ACK x ms`). Because the hub switches the siren on before sending the ACK, this time is an upper bound on the radio and processing part of the response. The total response time from a door contact opening to siren on is therefore:

$$t_response = t_debounce + t_event \rightarrow ACK = 50\ ms + t_event \rightarrow ACK$$

Table 8.1 Response time results (20 door events)

Quantity	Value
Radio round-trip time (HT6, ping test)	2.4–3.0 ms typical, 28.8 ms maximum
event → ACK, average	[to be filled]
event → ACK, maximum	[to be filled]
Total door-to-siren time, maximum	[to be filled] (requirement < 1 000 ms)

8.3 RANGE TEST

With the hub fixed on a desk, the detector was moved to 1 m, 5 m and 10 m in line of sight and to 10 m through one wall. At each position the door was opened and closed ten times and the number of events received, the number of retries and any offline indications were recorded.

Table 8.2 Range test results

Position	Events received (of 10)	Retries	Offline seen?
1 m, line of sight	[to be filled]	[to be filled]	[to be filled]
5 m, line of sight	[to be filled]	[to be filled]	[to be filled]
10 m, line of sight	[to be filled]	[to be filled]	[to be filled]
10 m, through one wall	[to be filled]	[to be filled]	[to be filled]

8.4 SYSTEM VALIDATION

Validation checks that the right system was built: every requirement from Chapter 3 is traced to at least one test (Table 8.4).

Table 8.3 Requirement-to-test traceability matrix

Requirement	Verified by	Result
FR1 Motion detection	ST5, HT2	[to be filled]
FR2 Door open/close	ST1, ST3	[to be filled]
FR3 Tamper in any state	ST6	[to be filled]
FR4 PIN arm/disarm (keypad and phone)	ST2, ST10	[to be filled]
FR5 Exit and entry delays	ST2, ST3, ST4	[to be filled]
FR6 Siren and status display	ST4, HT4, HT5	[to be filled]
FR7 Offline detector reported	ST7	[to be filled]
FR8 PIN lockout	ST8	[to be filled]

Requirement	Verified by	Result
NFR1 Event to siren < 1 s	Section 8.2	[to be filled]
NFR2 No lost events	ST1, ST11, HT6	[to be filled]
NFR3 Fail-safe	ST7, switch-to-GND input design	[to be filled]
NFR4 Range ≥ 10 m	Section 8.3	[to be filled]
NFR5 No internet needed	ST10 (hub's own access point)	Pass

8.5 DEMONSTRATION SCENARIOS

The final demonstration followed six scenarios that together exercise every requirement:

1. **Status at a glance:** OLED and phone page show DISARMED and detector online.
2. **Arm with exit delay:** PIN + A, 30 s countdown with chirps, then ARMED.
3. **Authorised entry:** door opens, 15 s entry delay, PIN + #, disarmed without siren.
4. **Intruder:** motion while armed causes an immediate alarm; disarm from the phone.
5. **Tamper:** detector lid opened while disarmed sounds the alarm.
6. **Sensor lost:** detector power removed; OFFLINE fault within 15 s.

[PHOTO PLACEHOLDER]

OLED showing EXIT DELAY and "Leave in NN s"

Fig. 8.2 OLED during the exit-delay countdown

[PHOTO PLACEHOLDER]

Hub with ALARM on the OLED, ALARM LED on and siren sounding

Fig. 8.3 System in the ALARM state

[PHOTO PLACEHOLDER]

Hub and detector serial monitors showing events, ACKs and event->ACK times

Fig. 8.4 Serial monitor logs during validation

8.6 DISCUSSION

The radio link proved fast and reliable on the bench: after start-up, 452 consecutive pings were answered with no loss and a typical round-trip time of under 3 ms. Since the debounce window of 50 ms dominates the response time, the total door-to-siren time is expected to be well below the 1 s requirement; the measured maximum is [to be filled]. The two-layer debounce converted heavily bouncing contacts into single clean events, and the fail-safe input wiring means that a cut wire produces an event rather than silence. Supervision through heartbeats turns a lost or destroyed detector into a visible fault instead of a silent gap in protection.

CHAPTER 9
OPERATION AND MAINTENANCE

9.1 USER GUIDE

Table 9.1 Everyday operation

Task	Keypad	Phone
Arm	Enter PIN, press A; leave within 30 s	Join HomeGuard-Hub, open http://192.168.4.1, enter PIN, tap Arm
Disarm / silence alarm	Enter PIN, press #	Enter PIN, tap Disarm
Correct a typing error	Press *	Edit the PIN box
Check status	OLED and LEDs	Web page (refreshes every 2 s)

When connecting a phone, the phone reports "connected, no internet"; this is expected because the hub's network is local only. Mobile data should be switched off so that the browser uses the hub's network, and the address must be typed with the `http://` prefix.

9.2 MAINTENANCE

- **Tamper alarm:** the alarm latches. Enter the PIN to silence it, then close the detector lid; the FAULT LED clears when the hub receives TAMPER_CLEAR.
- **Detector offline:** check the detector's power bank and the distance to the hub; the hub returns to ONLINE automatically when packets resume.
- **Changing the PIN or timings:** edit the constants at the top of `hub.ino` (for example `PIN_CODE`, `EXIT_DELAY_MS`) and re-flash the hub.
- **Replacing a board:** read the new board's MAC address with the L01 sketch, update `HUB_MAC` or `DET_MAC`, and re-flash both nodes.
- **PIR adjustment:** sensitivity and hold-time potentiometers on the module; allow 30 s warm-up after power-up.
- **Bench work inside the detector:** opening the lid triggers the tamper alarm by design; disconnect the buzzer lead during maintenance.

9.3 TROUBLESHOOTING

Table 9.2 Common faults and remedies

Symptom	Likely cause	Remedy
Detector logs SEND FAILED on every event	Hub not on channel 1 or wrong MAC	Check the hub prints channel 1; use the hub's AP MAC (STA MAC + 1) if needed

Symptom	Likely cause	Remedy
Cannot arm	Lid open or detector offline	Close the lid; wait for Detector ONLINE
Keypad ignored	PIN lockout active	Wait 60 s
Phone page does not load	Mobile data on or http:// missing	Turn mobile data off; type http://192.168.4.1
Alarm from PIR during testing	PIR sees the tester	Point the PIR away until needed
OLED blank, log shows OLED NOT FOUND	I2C wiring	Re-run the L05 I2C scan

9.4 KNOWN LIMITATIONS

- Only one detector is supported; the protocol already carries a node ID for future expansion.
- The ESP-NOW link is not encrypted, so packets could in principle be observed or spoofed.
- The PIN is fixed in firmware and is not stored in non-volatile memory.
- The detector is powered from USB; there is no battery monitoring or deep sleep.
- Alerts are local only (siren, display, web page); there is no SMS or cloud notification.
- The duplicate filter remembers only the last sequence number, which is sufficient for one detector with stop-and-wait delivery.

CHAPTER 10
CONCLUSION AND FUTURE SCOPE

10.1 CONCLUSION

A two-node wireless intrusion alarm was designed and built by following the embedded system design flow from requirements to operation and maintenance. A detector node with PIR, door and tamper sensing reports every event to a hub over ESP-NOW, and the hub decides, through a five-state alarm machine, whether to arm, delay, sound the siren or report a fault. Dependability was designed in rather than added later: fail-safe input wiring, two-layer debouncing, sequence-numbered packets with CRC, acknowledgements with retries, duplicate filtering, heartbeat supervision and PIN lockout.

Each design-flow stage produced a concrete artefact: a requirements list, a measurable specification, an architecture with a hardware/software partition, schematics with calculations, firmware, hardware and software test sheets with logs, a traceability matrix and a user guide. All artefacts, including the firmware and the step-by-step build notes, are kept in a public Git repository so that the system can be reproduced.

10.2 FUTURE SCOPE

- Support several detectors with per-node supervision and sequence tracking.
- Enable ESP-NOW encryption with a shared key to prevent spoofing.
- Battery-powered detector with deep sleep, wake-up on PIR or switch interrupts, and battery voltage reporting in the reserved packet field.
- PIN change from the keypad with storage in non-volatile memory, and RFID tags as an alternative to the PIN.
- Remote notification through GSM/SMS or a cloud service when an internet connection is available.
- A custom PCB and enclosure for both nodes.

REFERENCES

[1] P. Marwedel, *Embedded System Design: Embedded Systems Foundations of Cyber-Physical Systems, and the Internet of Things*, 4th ed. Cham, Switzerland: Springer, 2021.

[2] W. Wolf, *Computers as Components: Principles of Embedded Computing System Design*, 4th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[3] Espressif Systems, "ESP32 Series Datasheet," Shanghai, China. [Online]. Available: https://www.espressif.com

[4] Espressif Systems, "ESP-NOW," ESP-IDF Programming Guide, API Reference. [Online]. Available: https://docs.espressif.com

[5] Espressif Systems, "Arduino-ESP32 documentation," [Online]. Available: https://docs.espressif.com/projects/arduino-esp32

[6] "HC-SR501 PIR Motion Detector," module datasheet.

[7] "BISS0001 Micro Power PIR Motion Detector IC," datasheet.

[8] Solomon Systech, "SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller," datasheet.

[9] NXP Semiconductors, "UM10204: I2C-bus Specification and User Manual," Rev. 7.0, Oct. 2021.

[10] ON Semiconductor, "BC546B, BC547A, B, C, BC548B, C: Amplifier Transistors, NPN Silicon," datasheet.

[11] R. N. Williams, "A Painless Guide to CRC Error Detection Algorithms," 1993.

[12] Adafruit Industries, "Adafruit SSD1306 Arduino library," GitHub repository. [Online]. Available: https://github.com/adafruit/Adafruit_SSD1306

[13] M. Stanley and A. Brevig, "Keypad library for Arduino," Arduino Library Manager.

[14] J. G. Ganssle, "A Guide to Debouncing," The Ganssle Group, 2008.

[15] European Committee for Electrotechnical Standardization, *EN 50131-1: Alarm systems – Intrusion and hold-up systems – Part 1: System requirements*.

APPENDIX A
REPOSITORY STRUCTURE AND REPRODUCTION GUIDE

The complete project is kept in the Git repository github.com/linganiveth/esp32-wireless-intrusion-alarm. Its folders follow the design-flow stages used in this report.

Table A.1 Repository structure

Folder	Contents
docs/01_requirements … 08_operation_maintenance	One folder per design-flow stage with the stage document
docs/06_testing	Hardware and software test sheets and serial logs
docs/learning	Step-by-step build notes (concept, build, check) for each lesson
docs/daily_log	Day-by-day progress log and master checklist
docs/report, docs/slides	This report and the review presentation
hardware/schematics, bom, photos	Schematic PDFs, bill of materials, build photographs
firmware/detector_node, hub, common	Final firmware for both nodes and the shared packet definition
firmware/test_sketches	Unit-test sketches L01–L05 used during hardware bring-up
tools	Schematic generator, packet-header sync and latency analysis scripts

Steps to reproduce:

1. Install Arduino IDE 2.x, add the Espressif ESP32 board package (core 3.x) and select *ESP32 Dev Module*.
2. Install the libraries *Adafruit SSD1306*, *Adafruit GFX* and *Keypad* (Mark Stanley, Alexander Brevig) from the Library Manager.
3. Wire the hub and the detector according to the schematics in `hardware/schematics`.
4. Flash `firmware/test_sketches/L01_board_check` to each board and note its MAC address.
5. Put the hub's MAC address in `HUB_MAC` (detector_node.ino) and the detector's MAC address in `DET_MAC` (hub.ino).
6. Flash `firmware/detector_node` to the detector and `firmware/hub` to the hub; open both serial monitors at 115 200 baud.
7. Wait for `Detector ONLINE` on the hub, then run the tests in `docs/06_testing`.

8. Connect a phone to the Wi-Fi network *HomeGuard-Hub* (password homeguard123) and open http://192.168.4.1.

APPENDIX B
BILL OF MATERIALS

Table B.1 Core bill of materials (approximate Indian retail prices, October 2026)

#	Component	Specification	Qty	Amount (₹)
1	ESP32 DevKit V1 (ESP32-WROOM-32)	CP2102 or CH340 USB	2	900
2	Micro-USB data cable	Data, not charge-only	2	160
3	PIR motion sensor HC-SR501	5 V supply, 3.3 V output	1	90
4	MC-38 reed door sensor	Switch + magnet	1	70
5	Lever microswitch	Normally open	2	30
6	4×4 membrane keypad	8-pin	1	70
7	SSD1306 0.96" OLED	I2C, address 0x3C	1	180
8	Active buzzer	5 V	1	25
9	BC547 NPN transistor	—	3	6
10	1N4148 diode	Flyback (magnetic buzzer only)	2	6
11	5 mm LEDs	Green, yellow, red, blue	6	12
12	Resistors 10 kΩ / 1 kΩ / 330 Ω	1/4 W	10 / 5 / 6	21
13	Ceramic capacitor 100 nF	Code 104	10	20
14	ABS project box	~100 × 60 × 25 mm	1	80
15	Breadboards	830 + 400 points	2	150
16	Jumper wires and headers	M–M, M–F, 40-pin header	—	150
	Total (core)			≈ 1 970

The full list, including spares, optional items and tools, is in `hardware/bom/Home_Security_Component_List.xlsx`.